Assignment 2

Software Design

Author: Virag Raul

Group: 30433

1 Problem Definition

This project implements a Warehouse Management System - a web application that allows warehouse staff to manage product types, track physical stock units, create users, and perform inventory operations such as adding and searching items.

The system maintains two levels of inventory: a product catalogue (Items) and individual physical units (Stock Units), each identified by a system-generated serial number. A denormalized Stock counter provides a fast aggregate quantity of queries.

This document covers the analysis, design, implementation, and testing phases of the project.

2 Analysis Phase Use Cases & Requirements

2.1 Functional Requirements

- The system must allow users to add, edit, and delete product types (Items).
- The system must allow users to add stock units in bulk; each unit receives a system-generated serial number.
- The system must allow users to search stock units by product name or serial number and ordered ascending or descending.
- The system must mark individual units as Available, Reserved, Shipped, or Returned.
- The system must endure data exportation as a JSON/CSV/XML file to be downloaded.
- The system must send email after specific operation for the logged in Warehouse Account.

2.2 Use Cases

UC-1 Add Product Type

- Actor: Warehouse Admin + Seller
- Task: Creates a new Item with name, description, and base price.
- Result: Item persisted; visible in product list.

UC-2 Add Stock Units

- Actor: Warehouse Admin + Seller
- Task: Selects a product type and enters quantity; system creates many Stock Unit rows.
- Result: Stock counter incremented; units searchable by serial.

UC-3 Search Stock

- Actor: Warehouse Admin + Seller
- Task: Enters a product name or partial serial number in the search bar.
- Result: Filtered list of matching Stock Units displayed.

UC-4 Soft Delete operation -> only Admin, delete individual stock units and if no more shelf products left, it can also delete the Item category, and the stock associated with them.

Uc-5 Update operation -> both admin and seller on items and stock items

3 Design Phase

3. Architecture Overview

The application follows a three-layer architecture: Presentation (Blazor Server components), Business Logic (Service classes), and
Data Access (Entity Framework Core with SQL Server). The client browser communicates with the server over a persistent SignalR connection; UI updates are handled server-side and pushed to the browser as DOM diffs, with no direct database access exposed to the client. The exception is authentication (login/logout), which uses plain HTTP POST requests to Minimal API endpoints to correctly set and clear browser cookies, since cookies cannot be
set over SignalR.
This style isolates concerns: components hold no query logic;
services hold no markup, and repositories (via EF DbContext) are
Injectable and testable.

Entity-Relationship

The database contains three core tables:

- **Items** product catalogue. Attributes: Id, Name, Description, PricePerItem, IsDeleted, DeletedAtTime, LastModifiedTime.
- **StockUnits** one row per physical unit. Attributes: Id, ItemId (FK), SerialNumber, ActualPrice, Note, Status, IsDeleted.
- **Stocks** denormalized quantity counter. Attributes: Id, ItemId (FK), Quantity ≥ 0.

Items StockUnits: one-to-many. Items Stocks: one-to-one.

Component: Blazor Web Application

The Blazor Server application acts as the sole entry point. Razor components receive user interactions over a persistent SignalR connection, delegate to service classes, and re-render the affected parts of the UI server-side. A shared generic component (**GenericTable**) renders tabular data for any entity, keeping component code DRY.

Component: Service & Data Layer

GenericService<TEntity, TGet, TSend> provides default CRUD operations via EF Core. Concrete services (ItemService, StockUnitService) override methods that require navigation-property projections or business-rule validation + implement additional methods.

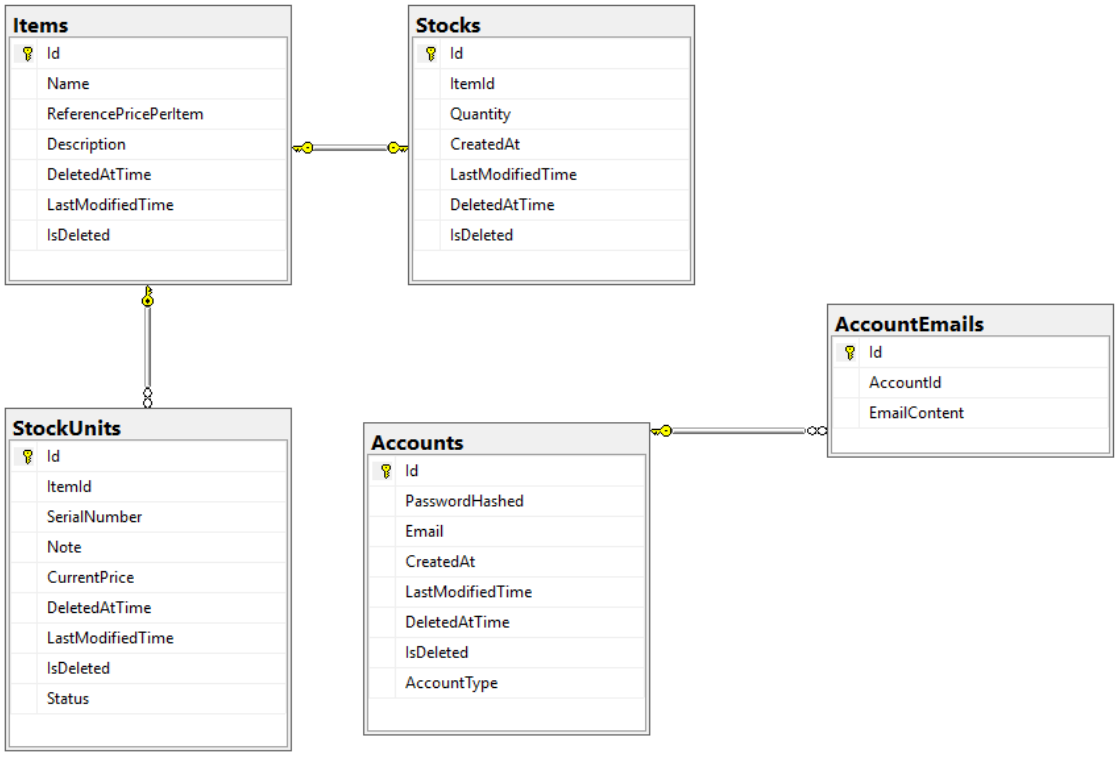
Design Patterns

- **Repository / Unit of Work** (structural) EF DbContext acts as the unit of work; DbSet<T> acts as repository. Decouples business logic from SQL.

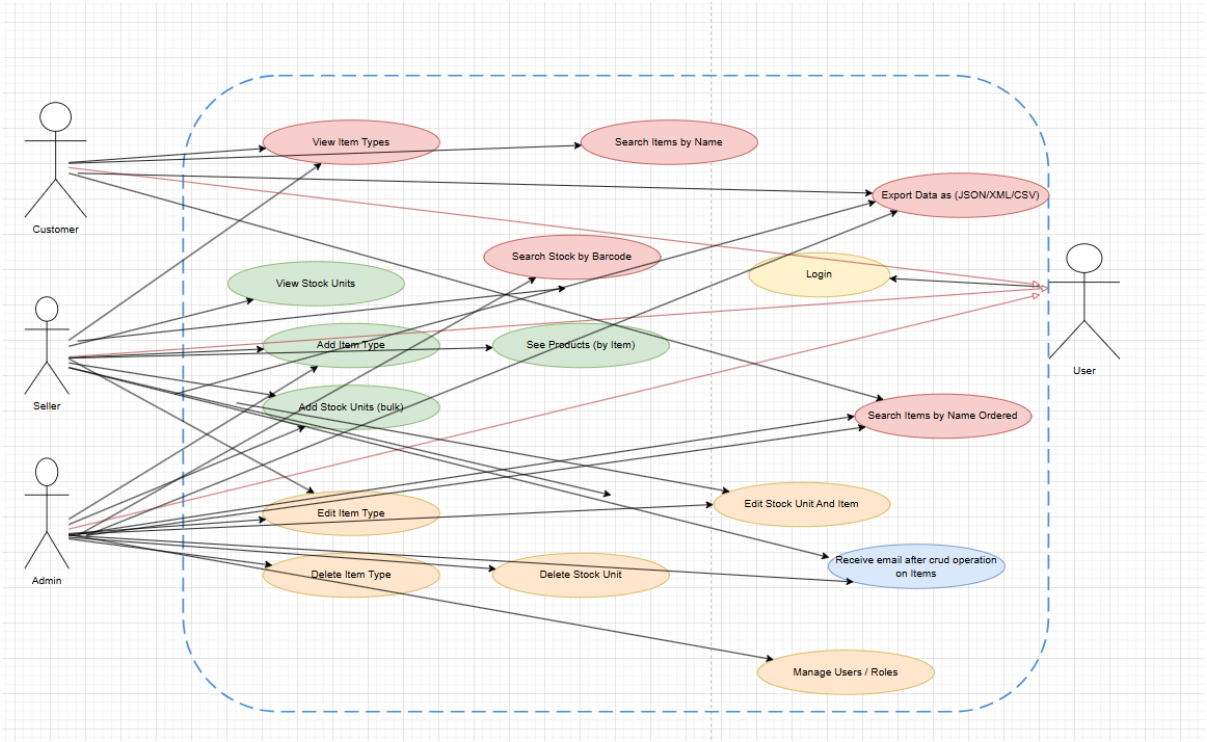
- **Generic Service** (behavioral) a single base class provides CRUD for any entity; subclasses override only what differs.

- **DTO / Mapper** (structural) AutoMapper translates between EF entities and lightweight DTOs, keeping the view layer decoupled from the data model.
- **Singleton** (creational) WarehouseEventBus and NotificationService are registered as singletons, ensuring a single shared instance exists for the lifetime of the application. Guarantees one event bus that all services are published.
- **Observer** (behavioral) WarehouseEventBus maintains an OnEvent delegate; services like ItemService publish events while NotificationService subscribes and reacts. Decouples the publisher from the consumer neither knows about the other directly.
- **Strategy** (behavioral) IExportStrategy defines a common export interface; ExportCSV, ExportJSON, and ExportXML are concrete implementations. The Export class delegates to whichever strategy is selected at runtime, making it easy to add new formats without changing existing code.

Entity-Relationship Diagram Warehouse System

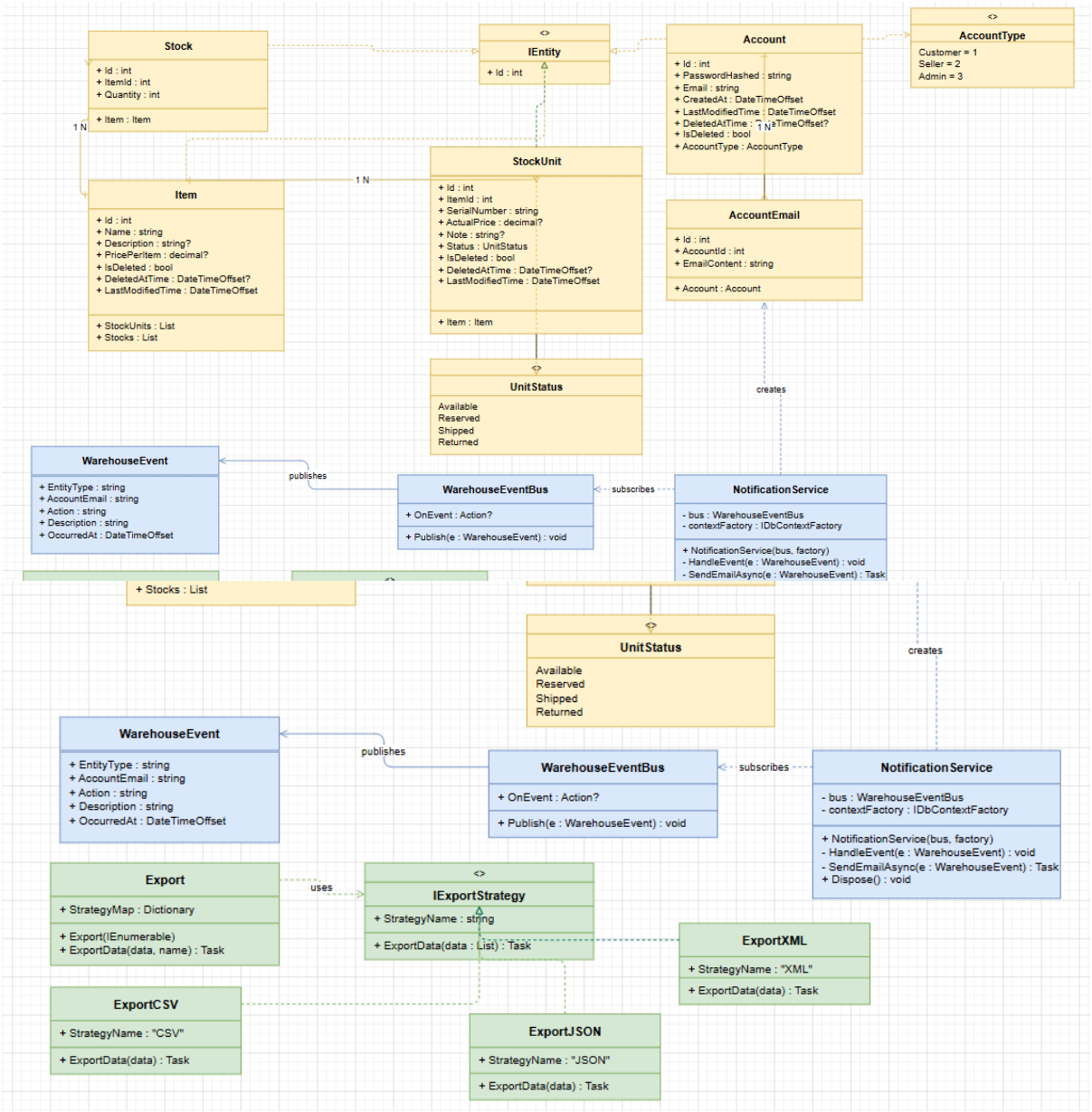


Use case Diagram

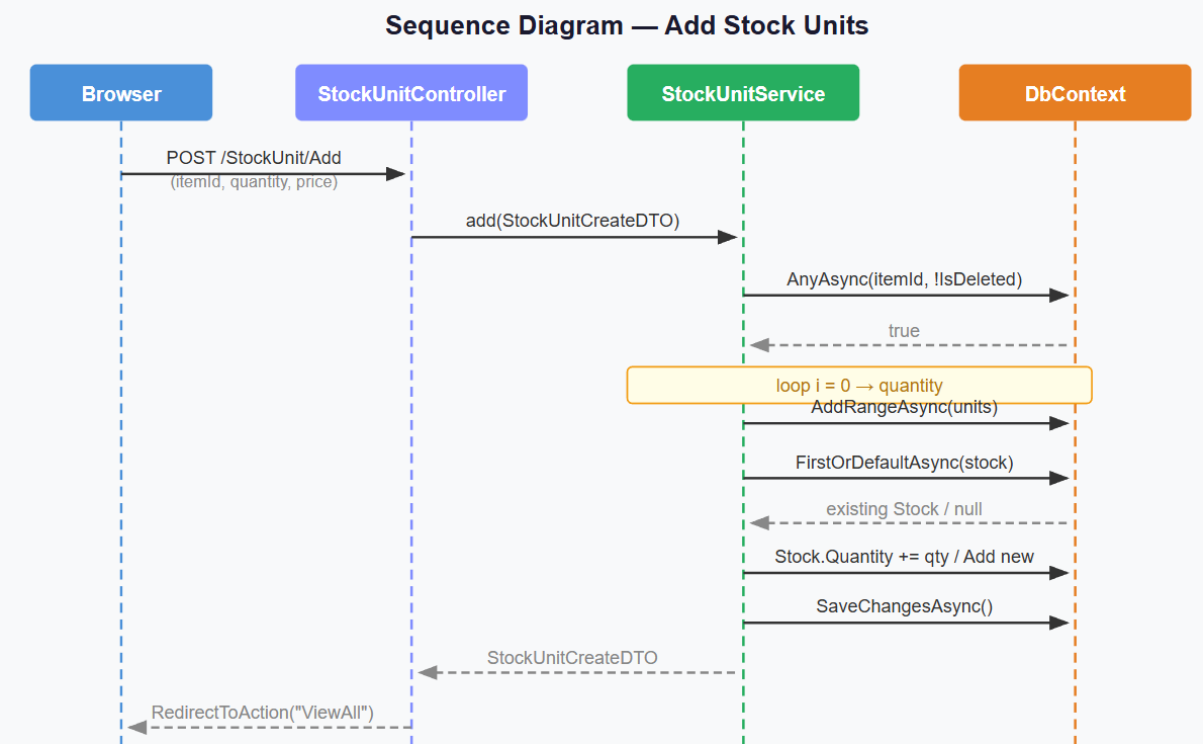


Core Domain Diagram

The entities and their relationships (Item, Stock Unit, Stock, Enums). It shows the structure of data in the app.



Sequence Diagram



4 Implementation Phase

4.1 Tools and Technologies

- C# / .NET 10 - primary language and runtime.
- ASP.NET Core Blazor Server web framework - providing component-based UI, SignalR-based interactivity, and server-side rendering.
- Entity Framework Core ORM with code-first migrations and LINQ query translation.
- SQL Server relational database - chosen for constraint support and EF Core compatibility.
- AutoMapper - object-to-object mapping between entities and DTOs.
- Bootstrap 5 framework for responsive UI.

4.2 Application Structure

- Warehouse.Data Entities, DTOs, DbContext, Migrations.
- Warehouse.Services GenericService base + ItemService, StockUnitService etc.
- Warehouse.Web (Blazor Server) Razor components, shared generic components (GenericTable), AutoMapper profiles, Minimal API endpoints for cookie authentication.
-

4.3 User Manual

- Navigation bar provides links to Items and Stock Units, visible based on the user's role. Key workflows:

- **Items View All:** lists all product types with Edit/Delete (Admin only) and a 'View Stock' link per row.
- **Items Add:** form to create a new product type (Admin only).
- **Stock Units View All:** lists all physical units with name and serial. Search bar accepts both product name and partial serial.
- **Stock Units Add:** select a product type and enter quantity; system generates serials automatically.
- **Login / Register:** accessible from the navigation bar when not logged in. On successful login the navbar updates to show the current user's email and a Logout button.
-

5 Testing and Validation

Validation was performed: manual integration tests via the UI.

5.1 Manual Integration Tests

- Navigation bar provides links to Items and Stock Units, visible based on the user's role. Key workflows:
- **Items View All:** lists all product types with Edit/Delete (Admin only) and a 'View Stock' link per row.
- **Items Add:** form to create a new product type (Admin only).
- **Stock Units View All:** lists all physical units with name and serial. Search bar accepts both product name and partial serial.
- **Stock Units Add:** select a product type and enter quantity; system generates serials automatically.
- **Login / Register:** accessible from the navigation bar when not logged in. On successful login the navbar updates to show the current user's email and a Logout button.